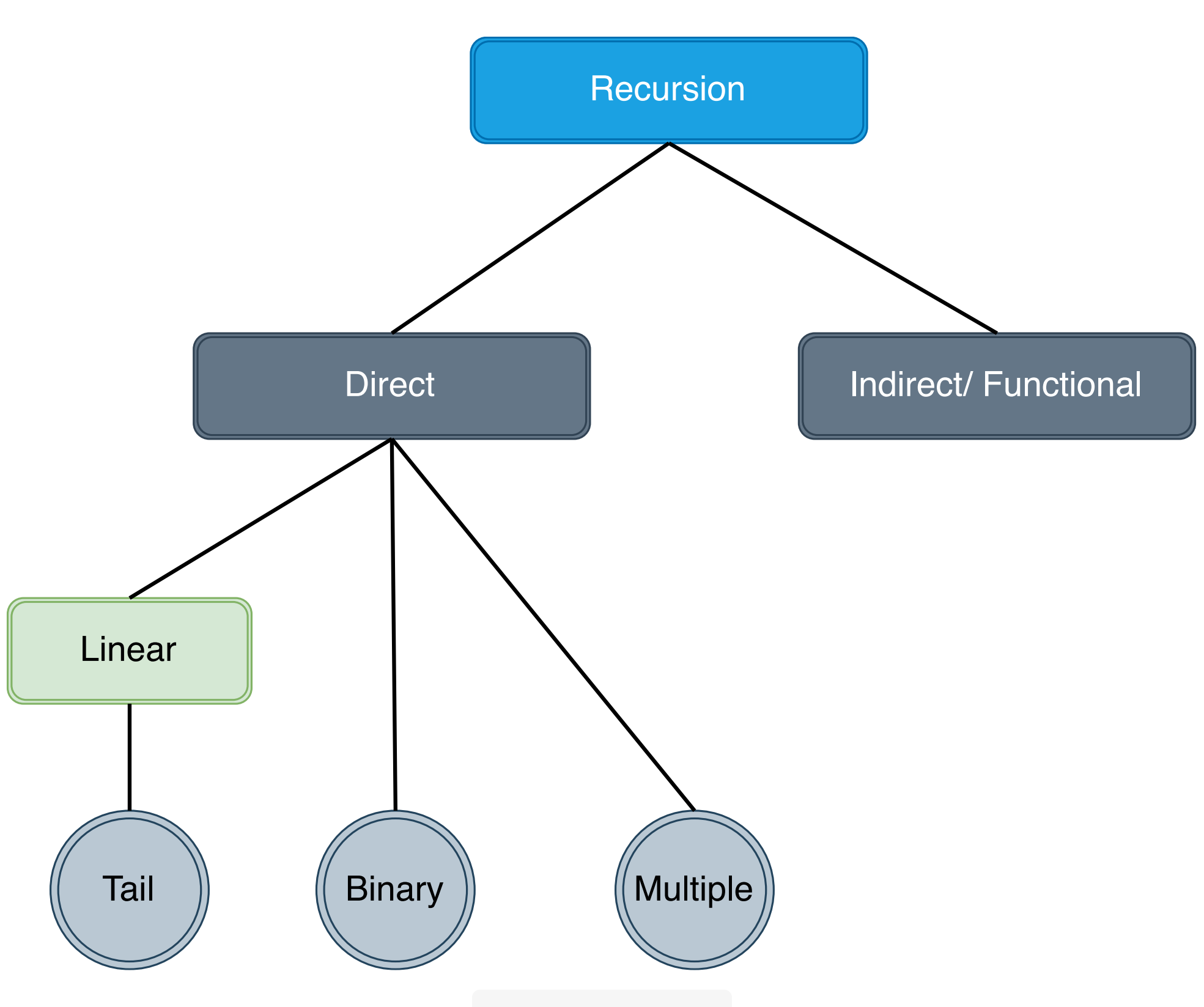
Recursion Types

Let's delve into the different types of recursion and their inherent characteristics.

Each type of recursion serves a specific computational paradigm, catering to diverse problem spaces. Grasping these variants enhances our competency in utilizing the recursive methodology in algorithmic design and problem-solving.

Here is a high-level view of recursion types:



Recursion Types

Let's start with the simplest type - the linear recursion.

Linear recursion

Linear recursion refers to a scenario where a function makes a single recursive call to itself. It is also known as single recursion. The simplest example of linear recursion can be a function that recursively prints from `1` to `n`.

Java

```java
public void printNumbers(int n) {
    if (n > 0) {
        printNumbers(n - 1);
        System.out.println(n);
    }
}
```

Here are some problems that can be solved using this recursion type:

- **Factorial computation:** Compute the factorial of a number `n` by multiplying `n` with the factorial of `n-1`.
- **Array summation**: Find the sum of elements in an array by recursively adding the current element with the sum of remaining elements.
- **String reversal:** Reverse a string by concatenating the last character with the result of reversing the rest of the string.
- **Linked list traversal:** Traverse a linked list by visiting the current node and recursively visiting the rest of the list.
- **Checking palindrome**: Check whether a string is a palindrome by comparing the first and last characters and then recursively checking the remaining substring.
- **Finding maximum in an array**: Find the maximum element in an array by comparing the current element with the maximum of the remaining elements.

Tail Recursion

Tail recursion is a subtype of linear recursion where the recursive call is the last operation in the function. In other words, a function with tail recursion has an empty epilogue.

Java

```java
public void printNumbersTail(int n) {
    if (n > 0) {
        System.out.println(n);
        printNumbersTail(n - 1);
    }
}
```

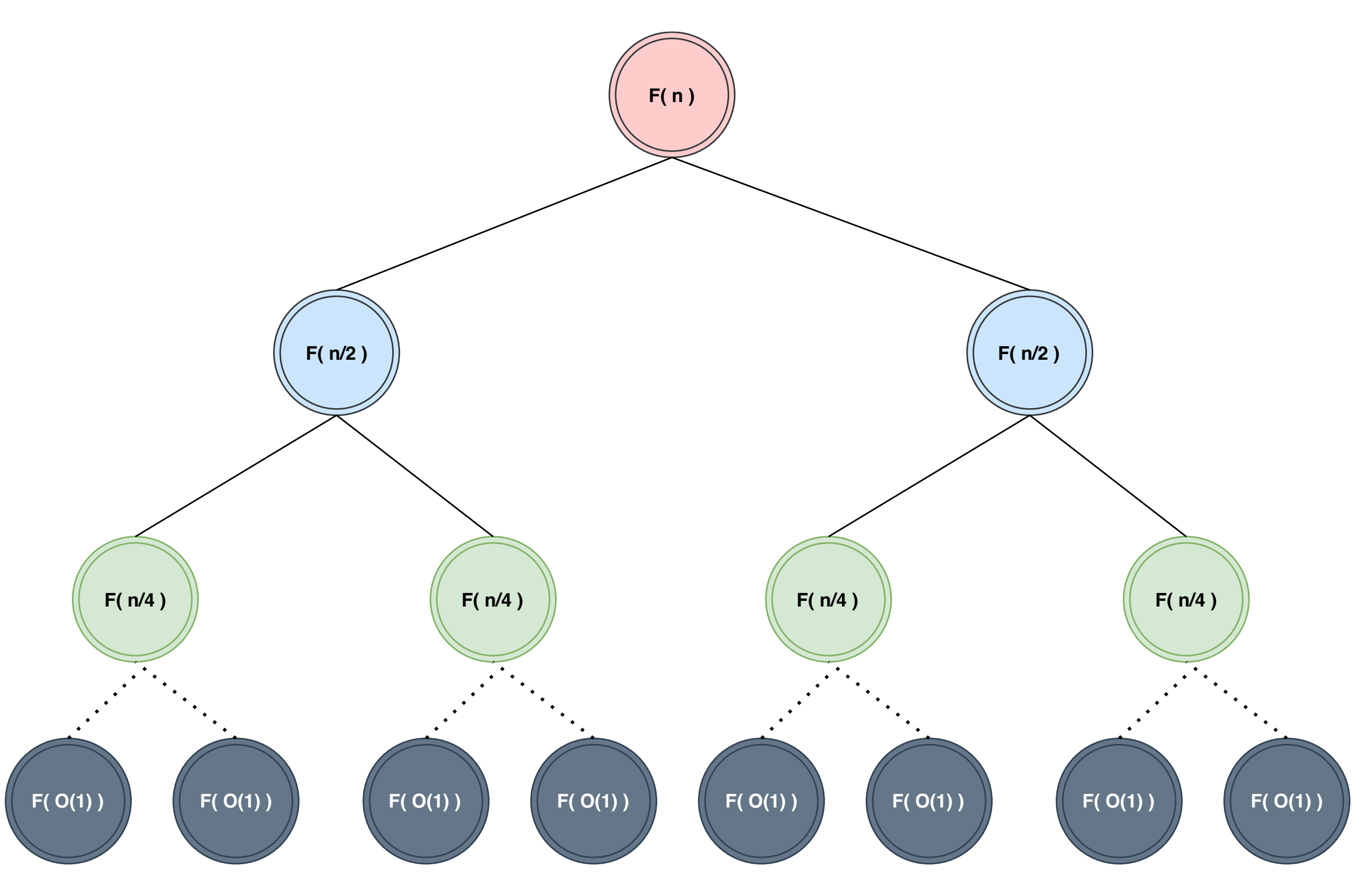
Here, `printNumbersTail` is a tail-recursive function. After the `print` operation, it directly calls itself.

Common problems solved using tail recursion are similar to those of linear recursion but are often optimized to be more efficient by taking advantage of the properties of tail recursion.

Binary recursion

A function exhibits binary recursion when it makes two recursive calls to itself. It is as if the function splits into two at each level, thus the name "binary" recursion.

For example, following illustration shows a recursion tree for a binary recursive function assuming that the `n` is the size for the problem. Additionally, it further assumes that each recursive call divides the `n` by half.



The above illustration is an ideal case for binary recursion. However, there can be cases where each recursive call, depending on the base case, can reduce/ increase the problem size in any manner. For example, let's have a look at the following `fibonacci()` recursive function:

Java

```java
public class Solution {

    public static int fibonacci(int n) {
        if (n <= 1) {
            return n;
        } else {
            return fibonacci(n - 1) + fibonacci(n - 2);
        }
    }

    public static void main(String[] args) {
        int n = 7;
        System.out.println("Fibonacci series has value: " + fibonacci(n) + " at term num
    }
}
```

Run Code Result Stdout

You must run your code first

Execute

The function `fibonacci()` calculates the `n`th Fibonacci number using binary recursion, making two recursive calls for each input greater than 1.

Many classical problems naturally map to this type of recursion. Some of the prominent ones are as follows:

- **Merge Sort**: Divide an array into two halves, sort each half independently, and merge them back together.
- **Quick Sort**: Partition an array around a pivot, then sort both partitions independently.
- **Binary Tree Traversals** (**In-Order, Pre-Order, Post-Order**): Each node requires visiting the left and right children independently.
- **Height of Binary Tree**: Determine the height of a binary tree by finding the maximum height of the left and right subtrees.
- **Maximal Subarray Sum**: Divide the array into two equal halves, and find the maximum sum in each half, then merge the results considering the case when the maximal sum subarray crosses the middle of the array.
- **Closest Pair of Points** Divide the given set of points into two halves, find the closest pairs in each half, and finally, find the closest pairs across the two halves.
- **Tower of Hanoi**: Solve by recursively moving n-1 disks to an auxiliary peg, moving the nth disk, and finally moving the n-1 disks again.
- **Karatsuba Algorithm for Fast Multiplication**: Split the digits of the numbers into two halves, and perform multiplications on the halves.
- **Skyline Problem**: Divide the buildings into two halves, solve for each half, and then merge the results.
- **Strassen's Algorithm for Matrix Multiplication**: Divide the matrices into four equal parts and recursively calculate the product for each pair of parts.

Multiple recursion

Multiple recursion, sometimes known as "multi-way recursion," is a technique where a function makes more than two recursive calls. This tactic typically applies to more complex problems that need to be broken down into a larger number of simpler, similar problems.

Let's explore a classic example of a problem that naturally lends itself to multiple recursion: traversing a ternary (3-child) tree. A ternary tree is a type of multiway tree where each node can have up to three children: left, middle, and right. To traverse this tree (say, in a pre-order fashion), you'd visit the root node first, then recursively traverse the left subtree, the middle subtree, and the right subtree.

Here's the pseudocode illustrating the multiple recursion:

```
function traverse(node):
    if node is not null:
        print(node.value)  // visit the node
        traverse(node.left) // go left
        traverse(node.middle) // go middle
        traverse(node.right) // go right
```

Here, the recursive `traverse()` function is called three times, once for each subtree. The recursion occurs multiple times within the same function context, which is why it's an example of multiple recursion.

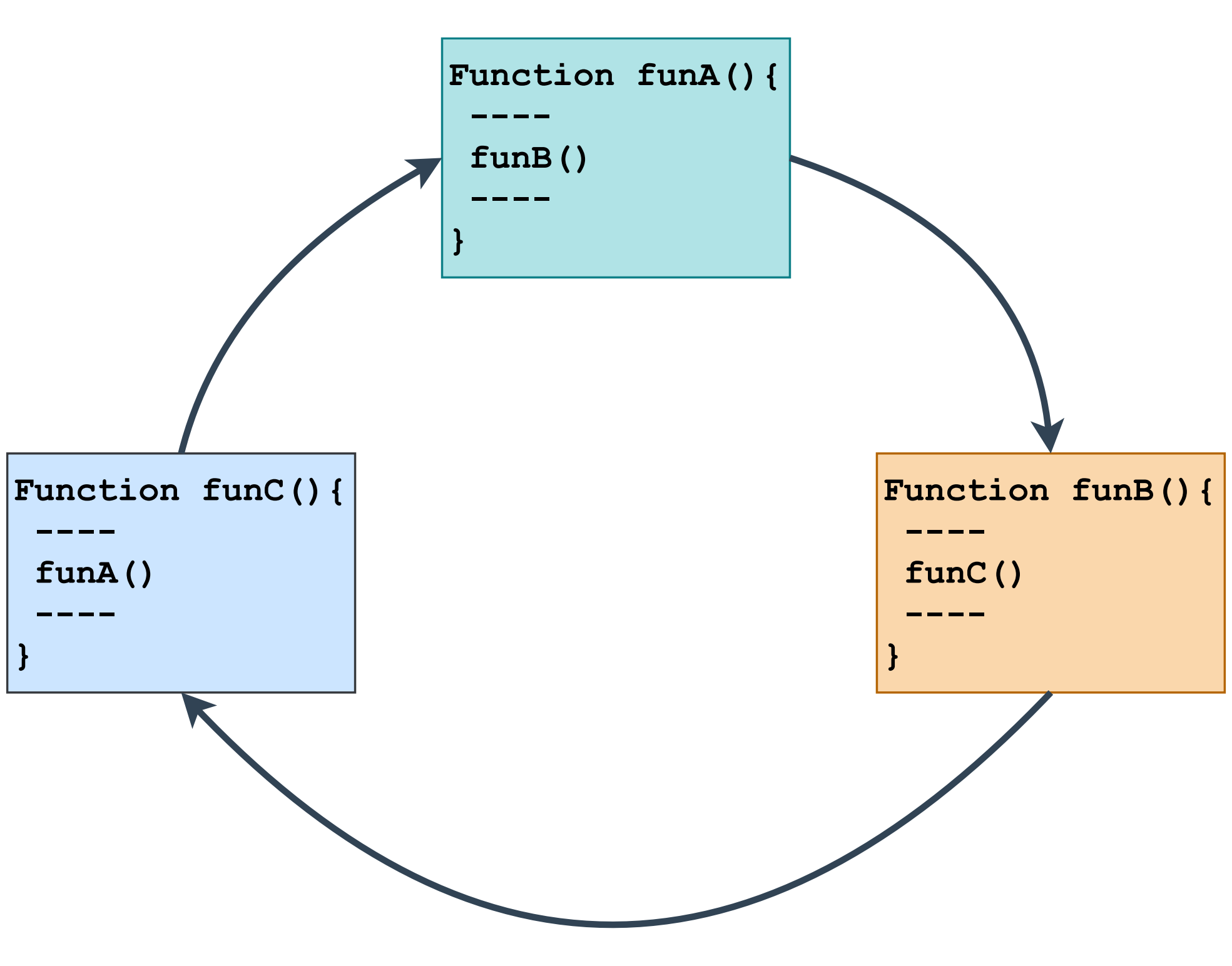
Multiple recursion often leads to elegant solutions for complex problems, but it can also result in high computational costs due to overlapping subproblems. This trait motivated the creation of Dynamic Programming, which optimizes recursion by storing and reusing the results of subproblems.

Here are a few examples that naturally require a solution following the multiple recursion type:

- **Sierpinski triangle:** A fractal design where each large triangle subdivides into 3 smaller ones in the next iteration, naturally leading to 3 recursive calls.
- **Fractal Tree:** Drawing a tree with multiple branches leads to multiple recursive calls - one for each branch.
- **Trémaux's algorithm**: Trémaux's algorithm leverages up to four recursive calls to explore all possible directions (North, South, East, West) from each cell in a maze, marking a successful path, and backtracking when a dead-end is encountered.
- **Flood Fill algorithm**: Given a screen, a point on the screen and a color, the Flood Fill algorithm colors the selected pixel and all adjacent (vertically and horizontally) pixels of the original color with the new color. It employs up to four recursive calls to fill a pixel and its adjacent pixels on a screen with a new color, replicating the fill tool in graphics editors.

Indirect/ functional/ multi-level recursion

Indirect recursion, also known as multi-level or functional recursion, occurs when a function is called not directly by itself but by another function that it calls, resulting in a cycle of function calls.



Consider an analogy of two friends, Alice and Bob, playing a game where Alice initiates a game move, then Bob makes his move in response, then Alice responds to that, and so on, until the game ends. The game moves of Alice and Bob can be thought of as functions `Alice()` and `Bob()` that indirectly call each other until a termination condition is met. Here's how this scenario can be translated into pseudocode:

```
Function Alice()
    If game not ended
        Make a move
        Bob()
    Else
        Declare result
    End If
End Function

Function Bob()
    If game not ended
        Make a move
        Alice()
    Else
        Declare result
    End If
End Function
```

Alice initiates the game by calling `Alice()`, which makes a move and then calls `Bob()`, and the cycle continues until the game ends. Multi-level recursion is a common theme in problems involving state machines, game theory, and grammar parsing.

Summary

The following table summarizes all recursion types.

Type	Calls	Key Feature	Pros	Cons
Linear	Single	One branch	Least computational, low memory usage	Limited to linear problems
Tail	Single	Last action	Can easily translated to the iterative equivalent	Less intuitive than other forms of recursion
Binary	Two	Split in half	Suits problems that naturally partition into halves	Potential for excessive recursive calls, can be memory-intensive
Multiple	Multiple	Complex splits	Can solve complex problems	High time complexity, hard to manage
Multi-level/Indirect	Multiple (via different functions)	Cross-function calling	Can solve complex multi-functional problems	Increased complexity, hard to debug